

课程笔记

Prefill vs Decode、KV-Cache、GEMM vs GEMV

基于公开课程资料整理

2025

Dive into Prefill / Decode

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 36 分钟

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言	2
2 Attention 的维度分析	2
2.1 Q, K, V 的维度	2
2.2 N 和 M 的差异	2
2.3 本章小结	2
3 Prefill 阶段	3
3.1 什么是 Prefill?	3
3.2 计算特性	3
3.3 本章小结	3
4 Decode 阶段	3
4.1 什么是 Decode?	3
4.2 计算特性	3
4.3 本章小结	4
5 KV-Cache	4
5.1 为什么需要 KV-Cache?	4
5.2 KV-Cache 的显存消耗	4
5.3 本章小结	4
6 GEMM vs. GEMV	4
7 系统调优：指标、硬件与服务策略	5
7.1 TTFT 与 TPS 的不同优化目标	5
7.2 为什么同一块硬件会出现“两张脸”	5
7.3 本章小结	5
8 部署核对清单	6
8.1 本章小结	6
9 总结与延伸	7
9.1 拓展阅读	7

1 引言

Prefill 和 Decode 是大模型推理部署中最基础也最核心的一组概念。随着 Apple M5 Max 芯片在这两个阶段上的性能突破，以及 SGLang 等推理框架的广泛使用，深入理解这两个阶段的计算特性变得尤为重要。

为什么要深入理解 Prefill vs. Decode ?

- 部署 VLM 时经常遇到这两个阶段的性能瓶颈
- SGLang 等框架的配置需要针对两阶段分别优化
- Apple M5 Max 等新硬件专门针对这两阶段做了优化
- 很多分析和介绍缺乏深度，本期追求更本质的理解

2 Attention 的维度分析

2.1 Q, K, V 的维度

回顾 Attention 的基本公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

维度分析：

- Q ：序列长度 N ，维度 d_k
- K, V ：序列长度 M ，维度 d_k (K) 和 d_v (V)
- K 和 V 的序列长度必须一致（因为 softmax 后的权重要乘以 V ）
- Q 的维度必须与 K 的维度一致（因为要算内积）

2.2 N 和 M 的差异

标准自回归注意力中 $N = M$ 。但存在两种场景使 $N \neq M$ ：

1. **交叉注意力** (Cross Attention)： Q 来自一个序列， K, V 来自另一个序列
2. **Decode 阶段**： Q 只有当前一个 token ($N = 1$)， K, V 包含所有历史 token (M 很大)

2.3 本章小结

维度分析是理解 Prefill 和 Decode 差异的基础。关键区别在于 Q 的序列长度：Prefill 时 N 很大，Decode 时 $N = 1$ 。

3 Prefill 阶段

3.1 什么是 Prefill ?

Prefill

用户发送 prompt 后，模型需要一次性处理所有输入 token。这个阶段叫做 **Prefill**：

- Q 的长度 = prompt 长度 N (通常几百到几千)
- K, V 的长度也是 N (因为是自注意力)
- 是一个大规模的 **矩阵-矩阵乘法** (GEMM)
- **计算密集型** (Compute-bound)

3.2 计算特性

Prefill 阶段的核心计算是 $Q \times K^T$ ，这是一个 $N \times d_k$ 乘以 $d_k \times N$ 的矩阵乘法，结果为 $N \times N$ 。

- 计算量: $O(N^2 \cdot d_k)$
- 数据量: $O(N \cdot d_k)$
- 计算/内存比 (Arithmetic Intensity) 很高 $\Rightarrow$ **计算密集型**
- GPU 的算力是瓶颈，内存带宽不是

3.3 本章小结

Prefill 是大矩阵乘法 (GEMM)，计算密集型，瓶颈在算力而非内存带宽。

4 Decode 阶段

4.1 什么是 Decode ?

Decode

Prefill 完成后，模型开始逐 token 生成输出。每步只生成一个新 token：

- Q 的长度 = 1 (只有当前 token)
- K, V 的长度 = 历史长度 M (包含所有之前的 token)
- 是 **向量-矩阵乘法** (GEMV)
- **内存带宽密集型** (Memory-bound)

4.2 计算特性

Decode 阶段的核心计算是 $q \times K^T$ ($1 \times d_k$ 乘以 $d_k \times M$)，结果为 $1 \times M$ 的向量。

- 计算量: $O(M \cdot d_k)$

- 但需要从内存加载整个 K 矩阵: $O(M \cdot d_k)$ 的数据量
- 计算/内存比 $\approx 1 \Rightarrow$ **内存带宽密集型**
- 内存带宽是瓶颈, GPU 算力严重闲置

4.3 本章小结

Decode 是向量-矩阵乘法 (GEMV), 内存带宽密集型, 瓶颈在内存带宽而非算力。

5 KV-Cache

5.1 为什么需要 KV-Cache ?

KV-Cache 的核心动机

在自回归生成中, 每个新 token 需要与**所有历史 token** 计算 Attention。如果每步都重新计算所有 K, V , 计算量为 $O(M \cdot d_k)$ (其中 M 不断增长)。KV-Cache 将历史 K, V 向量缓存在 GPU 显存中, 避免重复计算。

5.2 KV-Cache 的显存消耗

每层每个 token 的 KV-Cache 大小:

$$\text{Per-token KV} = 2 \times n_{\text{kv_heads}} \times d_h \times \text{bytes}$$

总 KV-Cache:

$$\text{Total KV} = L \times M \times 2 \times n_{\text{kv_heads}} \times d_h \times \text{bytes}$$

其中 L 是层数, M 是序列长度。这就是为什么 GQA (减少 KV heads) 对推理效率如此重要。

5.3 本章小结

KV-Cache 是自回归推理的核心优化, 避免了重复计算 K, V 。它的显存消耗随序列长度线性增长, 是长上下文推理的主要瓶颈。

6 GEMM vs. GEMV

表 1: Prefill vs. Decode 计算特性对比

	Prefill	Decode
核心操作	GEMM (矩阵 $\times$ 矩阵)	GEMV (向量 $\times$ 矩阵)
Q 长度	N (prompt 长度)	1
瓶颈类型	计算密集型	内存带宽密集型
GPU 利用	算力充分利用	算力严重闲置
优化方向	更强的 FLOPs	更大的内存带宽
延迟特征	TTFT (首 token 延迟)	TPS (每秒 token 数)

Decode 阶段的效率困境

Decode 阶段每步只生成一个 token，但需要加载整个模型权重和 KV-Cache。GPU 的算力大量闲置，内存带宽成为瓶颈。这就是为什么 Apple M5 Max 的统一内存架构（高带宽）在 Decode 阶段表现突出。

7 系统调优：指标、硬件与服务策略

7.1 TTFT 与 TPS 的不同优化目标

从服务视角看，Prefill 和 Decode 对应着两类完全不同的用户体验指标。Prefill 更直接影响首 token 延迟 (TTFT)，决定“用户是否觉得系统响应够快”；Decode 更直接影响每秒输出速度 (TPS)，决定长回答时的体感流畅度。理解两者差异，才能正确设置批处理、并发度和硬件预算。

表 2: 推理服务中最常见的两类目标

指标	更受哪个阶段影响	典型优化手段
TTFT	Prefill	提高 GEMM 吞吐、减少 prompt 冗余、提前编译 kernel
TPS	Decode	提升内存带宽、优化 KV cache、增大 decode batching
成本/吞吐比	两者共同作用	调整并发、batch、上下文长度与量化策略

7.2 为什么同一块硬件会出现“两张脸”

一块 GPU 在 Prefill 阶段可能表现得像“算力怪兽”，在 Decode 阶段却像“带宽受限的搬运工”。这并不是硬件突然变差，而是工作负载结构发生了变化。课程的真正启发在于：评估推理硬件时，不能只看单一 FLOPs 指标，还要看带宽、缓存层次和访问模式是否匹配 Decode 的内存密集特征。

部署时最容易被忽略的一点

很多团队只看 benchmark 平均吞吐，却忽略了 prompt 长短分布和用户请求类型。如果业务以长 prompt、短回答为主，Prefill 可能是主瓶颈；如果业务以短 prompt、长回答为主，Decode 的带宽瓶颈会更加突出。

7.3 本章小结

Prefill/Decode 的差异不只是算法知识，而是服务架构设计知识。只有把指标、硬件和工作负载类型放在一起看，推理调优才不会跑偏。

8 部署核对清单

真正把课程里的分析落到服务侧，团队通常要在线上回答几个问题：当前业务的 prompt 长短分布是什么？主要优化 TTFT 还是 TPS？KV cache 是否会在高并发时触发显存抖动？这些问题如果不提前梳理，往往会让系统在真实流量下暴露出与 benchmark 完全不同的瓶颈。

上线前至少核对四件事

- 用户请求以长 prompt 还是长回答为主
- 现有硬件瓶颈更偏向算力还是带宽
- KV cache 和 batch 策略是否匹配目标并发
- 监控面板是否同时覆盖 TTFT、TPS 与显存占用

8.1 本章小结

Prefill/Decode 的理论差异最终都会回到部署清单上。只有把工作负载、硬件和指标统一到一个核对流程里，推理优化才真正可执行。

9 总结与延伸

1. Prefill = GEMM = 计算密集型; Decode = GEMV = 内存带宽密集型
2. KV-Cache 避免了 Decode 时重复计算 K, V , 但消耗大量显存
3. GQA 通过减少 KV heads 压缩 KV-Cache, 是推理效率的关键
4. 后续将介绍 MHA (Massive Head Attention) 和 KiMi 发布的 Attention Allocation 等新工作

9.1 拓展阅读

- Apple M5 Max 的统一内存架构对 LLM 推理的影响
- SGLang: 高效 LLM 推理框架
- FlashAttention: 加速 Prefill 阶段的 Attention 计算
- Paged Attention: 优化 KV-Cache 的显存管理